Online Movie Theater Ticketing System

Software Requirements Specification

<1.0>

<February 12 2026>

<Group 10>
<Justin Nguyen, Christopher Lopez, Yusuf Qargha>

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Github link:

https://github.com/jnguyen1748/CS-250-Movie-Theater-Project-

Revision History

Date	Description	Author	Comments

<2026/02/12>	<Version 1>	<Justin Nguyen, Christopher Lopez, Yusuf Qargha >	Placing foundation
<2026/03/05>	<Version 1.1>	<Justin Nguyen, Christopher Lopez, Yusuf Qargha >	Added architectural and uml class diagrams, with explanations for each.
<2026/03/19>	<Version 1.2>	<Justin Nguyen, Christopher Lopez, Yusuf Qargha >	Added planning for testing of the project.
<2026/04/10>	<Version 1.3>	<Justin Nguyen, Christopher Lopez, Yusuf Qargha >	Added final architeht

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
Justin Nguyen	Justin Nguyen	Software Eng.	2026/03/05
	Dr. Gus Hanna	Instructor, CS 250	
Christopher Lopez	Christopher Lopez	Software Eng.	2026/03/05
Yusuf Qargha	Yusuf Qargha	Software Eng.	2026/03/05

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4

A. Appendices 5

A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

1.1 Purpose

The purpose of this document (abbreviated as SRS) is to provide a thorough description of the planned online ticketing system for the local movie theater requesting this project. Specifically, this SRS is intended to be read by the developers planning to implement the described product, any management with a stake in this product, and any QA/testers involved in the success of this product.

1.2 Scope

PRODUCT NAME: Online Movie Theater Ticketing System (OMTTS or TTS)
The overall goal of this product is to be a web-based ticketing system for use with a local movie theater. Customers will be able to browse movies being screened and securely purchase tickets, as well as create an account for access to these features.

All planned product features:

User and Account management
Objective: provide access to important information for users as well as simplify certain actions, such as future purchases.

- An account will store the following information:
- Payment information for faster ticket purchases
- Purchase history and access to the tickets bought for each specific purchase

Online ticket purchasing system
Objective: Allow users to consistently purchase tickets online and easily customize options for their tickets.

- Customers will be able to purchase tickets for specific showtimes, as well as for any available seats of their choice.

Secure and simple payment
Objective: allow users to purchase tickets online using various payment options, all done as securely as possible.

- Protect user payment information during processing using a secure payment protocol
- Support multiple payment methods, such as PayPal, Apple Pay, credit cards, etc.

Security and anti-fraud/bot measures
Objective: Implement thorough measures to deal with bots purchasing mass amounts of tickets, as well as prevent bad actors from making fraudulent purchases.

- Ensure all tickets are unique and cannot be illegally duplicated or resold.
- Maintain system integrity and general security through alerts to suspicious activity, as well as rate limiting and captcha verification for the prevention of bot access.

Log valuable data and implement administrator systems to easily access this info
Objective: Collect and store useful information for future use, and have an easy way for employees and managers to view and edit this information.

- Log business-relevant information, such as ticket sales, along with information about each ticket purchased, such as the showtime purchased and chosen seat.
- Administrators should be able to view this information easily and export it to a variety of formats for use in future business decisions.

Customer engagement
Objective: Allow customers to easily report bugs, issues, or suggest features to improve this product in the future.

- Ensure that such customer feedback is easy for customers to submit.

Simple and eye-catching user interface
Objective: keep the UI intuitive and easy for users to navigate, while also being visually appealing. Also ensure the UI will correctly function on mobile devices.

- Clearly label sections and ensure anyone can easily figure out what features are where.
- Include accessibility features for the product to be used by anyone.

Digital and physical ticket options
Objective: support both digital and physical tickets.

- Digital tickets can be presented by a person using a device and accepted by the theater.
- Tickets can also be printed out and presented to staff at the theater.

1.3 Definitions, Acronyms, and Abbreviations

UI: User Interface. This refers to the means by which the user interacts with the webpage, including, but not limited to, buttons, menus, and links.
OMTTS: abbreviation for the product name, Online Movie Theater Ticketing System.
TTS: a shorter abbreviation for the product name.
SRS: abbreviation for this document's official name, Software Requirements Specification
ADMINISTRATOR: An account with full system control that is intended to be used by employees or management.
USER: a customer with an account using the product.

1.4 References

NOT NEEDED AT THE MOMENT (NO OUTSIDE DOCUMENTS ARE REFERENCED FOR THIS VERSION OF THE SRS)

This subsection should:
(1) Provide a complete list of all documents referenced elsewhere in the SRS, or in a separate, specified document.
(2) Identify each document by title, report number - if applicable - date, and publishing organization.

(3) Specify the sources from which the references can be obtained.
This information may be provided by reference to an appendix or to another document.

1.5 Overview

The rest of this document will go into further detail about all functional and non-functional requirements needed for this product to be completed, as well as any other necessary information pertaining to the implementation of this product. Each section contains the following information:

1. **Introduction:**
   The purpose of this product, as well as the general scope of the final product are discussed here. Important information pertaining to understanding the rest of this document is also listed in this section.

2. **General Description:**
   This section goes over general factors that will affect the product. It also goes over the restraints pertaining to the implementation of this product.

3. **Specific Requirements:**
   Very specific details about the technical requirements of this product are listed here. The general planning for the implementation of this product is also listed in this section.

4. **Analysis models:**
   In progress

2. General Description

This section is for a description of certain factors that will affect the product and requirements that must be considered for the completion of this product. Specific requirements are not listed here, but rather general and broad factors/requirements will be listed in this section.

2.1 Product Perspective

The TTS is an online product intended to support ticket sales and provide data for future decisions for the local movie theater. Existing systems, such as ticket sales, data collection, and movie scheduling, will be seamlessly integrated so that the product can work seamlessly with existing systems in place.

It should be noted that the TTS will serve as the primary online function for the local movie theater and will be designed to work on mobile and PC devices. Any additional future online-based endeavors should also be designed to work in conjunction with this product or be developed as extensions to this product.

In terms of integration with existing systems, tickets sold through the product will need to be able to integrate with existing in-person ticketing and theater entry systems. Physical theater operations will work alongside this product. Data such as seat preferences and ticket purchasing trends will be appropriately exported in convenient formats to work alongside existing data already logged.

2.2 Product Functions

The following major functions are planned to be implemented in the product:

User accounts with the ability to store tickets for convenient access, past purchase history, and save payment methods for future use.

Movie and showtime browsing, all in an easy-to-browse system, while being pleasing for customers to browse. Available seating will also be easily viewable at first glance.

Ticket purchasing. Users should be able to select a specific movie screening time, as well as a specific seat, and purchase a ticket corresponding exactly to their selected options.

Secure payment processing. Also support a wide variety of payment options, all while maintaining integrity and preventing fraud.

Site security and anti-fraud/anti-bot measures. This feature will also ensure that each ticket is unique and cannot be resold or duplicated.
Administrative tools for easy management of users and easy viewing of collected information and data.

2.3 User Characteristics

In addition to regular customers, employees and management will also be expected to use this product.

Regular customers who are primarily seeking to purchase tickets will be the main target of this product. A regular customer should be expected to have basic technical knowledge and should be familiar enough to navigate through the site with little help. However, a complicated site structure will not be ideal for the average customer, and thus, a simple user interface is heavily stressed as an important feature. Accessibility features will be necessary for disabled individuals; however, said accessibility features will be for physical disabilities primarily, such as color blindness.

Employees will be expected to use the site for the management of the site, such as updating the site with movie showings, monitoring ticket sales, and viewing users for troubleshooting.

Features intended for employee use will be simply designed but will not be designed with visual appeal in mind. Employees will also be expected to be taught how to use these features. Management will also have the same technology literacy as employees, but will be primarily using the site for data for future business decisions. Features designed for management will be of a similar design to features aimed at employees.

2.4 General Constraints

The following constraints limit the design and implementation of the Online Movie Theater Ticketing System (OMTTS):

1. The system is implemented as a web-based application accessible through modern web browsers.
2. System shall comply with applicable payment security standards when processing transactions.
3. Shall be integrated with the theater's in-person ticket validation and entry system.
4. Shall operate on standard commercial hosting infrastructure and shall not require specialized hardware.
5. System will comply with applicable data protection and privacy regulations governing customer information.
6. The system shall support both desktop and mobile device access without requiring the installation of additional software.

2.5 Assumptions and Dependencies

Assumptions and dependencies that affect the requirements:

1. The movie theater has reliable internet connectivity to support online ticket transactions.
    a. Customers accessing the system will have internet-enabled devices such as phones, tablets, and or personal computers.
2. The movie theater will provide accurate movie schedules, showtimes, and seating layout data prior to system configuration.
3. Third-party payment processing services (PayPal, Master Card, Visa, and Apple/Google Pay) will provide stable and accessible APIs.
4. An email service provider will be available to send confirmation emails and password reset notifications.
5. In person theater ticket scanning system will support validation of digitally generated QR codes or barcodes.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Online Movie Theater Ticketing System will provide a web-based interface accessible through modern browsers on both desktop and mobile devices. The homepage will display

available movies and showtimes, allowing customers to select a movie and proceed to the seat selection page. The seat selection interface will clearly indicate available, reserved, and sold seats.
The system will include account features for user registration, login, and personal information management. Logged-in users will have access to a dashboard where they can view their purchase history, access digital tickets, and manage stored payment methods. During checkout, users will review selected tickets and complete payment through a secure payment page. After a purchase, a confirmation email with the user's details and digital ticket(s) will be sent to the user, and digital ticket(s) will be accessible through the user account.
A separate administrative interface will allow authorized employees to manage movie listings, update showtimes, and view ticket sales data. All interfaces will be responsive across devices and include accessibility features such as clear text and proper color contrast.

3.1.2 Hardware Interfaces

The system is intended to work with standard consumer devices such as phones, tablets, laptops, and desktop computers. Customers will not need any special hardware beyond an internet-enabled device with a supported browser. For theater entry, the system will generate digital tickets containing a QR code or barcode that can be scanned using the theater's in-person ticket scanning equipment. Customers will also have the option to print their tickets using a standard printer if they prefer a physical copy for entry.

3.1.3 Software Interfaces

The Online Movie Theater Ticketing System will connect with third-party payment processing services in order to securely complete financial transactions. These services will handle payment verification and authorization for supported payment methods. The system will also integrate with an email service provider so that users can receive purchase confirmations, digital tickets, and password reset notifications. Additionally, the system will work alongside the theater's in-person ticket validation system to ensure that tickets purchased online can be properly verified at the time of entry. All user accounts, ticket data, and transaction records will be stored within a structured database system to maintain organization and data integrity.

3.1.4 Communications Interfaces

All communication between users and the system will occur over secure internet connections using HTTPS to protect sensitive information. This includes login credentials, personal information, and payment data.
When communicating with external services such as payment processors and email providers, the system will use secure API connections to ensure data is transmitted safely. Email communications will also be sent using secure transfer protocols to maintain user privacy.

3.2 Functional Requirements

3.2.1 MOVIE TICKET PURCHASING

3.2.1.1 Introduction
BuyMovieTicket(,){
 ……
}
3.2.1.2 Inputs
User ID who has a credit card / or third-party money system already on the account

3.2.1.3 Processing
Check the credit card to be valid, or the third-party money authenticator, such as paypal is open and valid.
Check the USER ID to make sure the account is valid.
Check the age if the movie is age-appropriate for the user.
Check if the user's funds are sufficient enough for the movie ticket.
Check movie date and time if available.
Check the seats of the movie to determine whether or not seats exist.
Process the information of the purchase to be stored somewhere.

3.2.1.4 Outputs
Ability to print out a physical ticket
An online ticket
Message displaying purchase details
Payment is stored in the user's purchase history and is available to be seen.
Ability to refund whenever possible.

3.2.1.5 Error Handling
If the user doesn't have a credit card or a way to give money, reject the purchase.
If the user enters a large sum of money, reject the request for the sake of security.
If the user purchases a movie ticket and buys another movie ticket that has overlapping times, reject the request.
If the user is too young to purchase a movie ticket, reject the request.

3.2.2 ticket refund

3.2.2.1 Introduction
refundTicket()

3.2.2.2 Inputs
User ID and Ticket Number

3.2.2.3 Processing
Identify the user's ID, make sure it is valid, and check if the user's ticket exists and is valid. Additionally check the time of the refund. The cost of the ticket will also be processed.

3.2.2.4 Outputs
If the user's refund is successful, display a message with the info of the refund, including ticket id and amount refunded.

3.2.2.5 Error Handling
If the user requests a refund after the movie has already started, reject the refund. If the movie ticket doesn't exist or the user doesn't have a way to store money on their USER ID (ie paypal, Apple Pay, credit card, etc.), then reject the refund.

3.3 Use Cases

3.3.1 Use Case 1: purchasing tickets

Name: Purchase Ticket
Actor(s): Customer

FLOW OF EVENTS:

1. The customer will log into the TTS
2. The system will authenticate the customer and verify that they are a human user.
3. The customer will browse the available movie showings and select a desired showtime
4. The customer will select an available seat from the list of available seats
5. The customer will proceed to online payment and pay for the ticket using their preferred payment method
6. System processes payment
7. If payment succeeds, a unique ticket is generated for the Customer, which the Customer can choose to print out

ENTRY CONDITIONS:

- The customer will be required to have and use a user account to properly purchase tickets.
- The System the user is using must be capable of supporting a browser capable of accessing the TTS.
-

EXIT CONDITIONS:

- NO ACCOUNT: The user will be redirected to the user creation page to create an account.
- Successful Completion
    - A unique ticket ID is generated
    - Transactions are recorded in the database.
    - Confirmation message is displayed.
    - Confirmation email is sent.
- Failure
    - Seats are released if payment fails/session expires.
    - System displays an appropriate error message.

3.3.2 Use Case 2: Creating Accounts

Name: Create an Account
Actor(s): Customer

FLOW OF EVENTS :

1. Customer navigates to the account creation page
2. Customer enters personal information for their account (name, email, password)
3. System validates email exists, and the password is strong enough (length and special characters)
4. The system creates a new account in the database
5. The system sends a confirmation email to the customer

ENTRY CONDITIONS:

- The System the user is using must be capable of supporting a browser capable of accessing the TTS.

EXIT CONDITIONS:

- NO ACCOUNT: The user will be redirected to the user creation page to create an account.

3.3.3 User feedback

Name: Submit Feedback
Actor(s): Customer, Administrator

Purpose: Allows customers to easily report bugs, suggest new features, or provide general feedback in order to improve the Movie Theater Management System.

FLOW OF EVENTS:

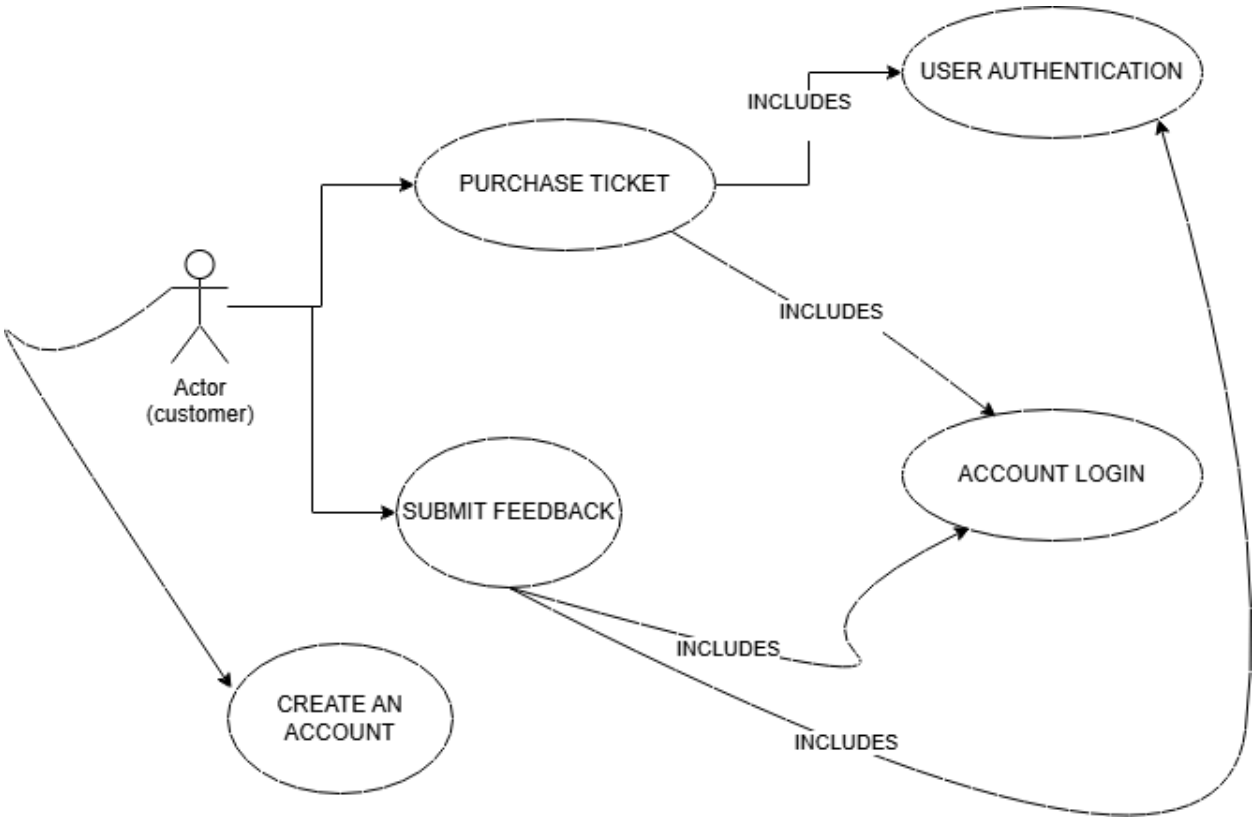
1. Customer navigates to the Feedback/Bug Report page
2. System displays a feedback submission form.
3. Customer enters feedback text describing the issue or suggestion
4. Customer selects feedback category (Bug, Feature Request, General Comment)
5. Customer optionally attaches screenshot(s) to provide additional context.
6. Customer enters email address (required if follow up response is requested)
7. Customer clicks Submit.
8. System validates required fields (feedback text and email format)
9. System checks file size and format of attachments
10. System stores feedback in the database
11. System generates a unique Feedback ID
12. System displays confirmation message to customer
13. System sends confirmation email
14. Customer is prompted to return to the homepage

ENTRY CONDITIONS:

- The customer will be required to have and use a user account to properly submit feedback.
- The System the user is using must be capable of supporting a browser capable of accessing the TTS.
- Customer has access to an active internet connection.

EXIT CONDITIONS:

- NO ACCOUNT: The user will be redirected to the user creation page to create an account.
- Successful Completion:
- Feedback is stored in the system database.
- A unique feedback ID is generated.
- A confirmation message is displayed to the customer.
- Confirmation email is sent (if email provided).
- Failure:
- Feedback is not stored due to outside factors (internet going out, page refreshed)
- Person does not fill out all required fields



3.4 Classes / Objects

3.4.1 MOVIE

3.4.1.1 Attributes
MOVIE NAME
MOVIE DATE
MOVIE TIME
MOVIE COST
MOVIE ID
MOVIE ROOM SEATS
MOVIE DURATION
MOVIE RATINGS
MOVIE AGE REQUIREMENT
3.4.1.2 Functions
Functions to check the age verification of the movie as well as return crucial information such as the date, time, cost, etc.

UML Diagram 1:

Movie
movieName : String movieDate : int movieTime : int movieCost : double movieID : int movieRoomSeats : boolean[][] movieDuration : int movieRating : double movieAgeRequirement : int
getMovieName() : String getMovieDate() : int getMovieTime() : int getMovieCost() : double getMovieID() : int getMovieRoomSeats() : boolean[][] getMovieDuration() : int getMovieRating() : double checkAgeRating(userAge : int) : boolean getMovieDetails() : String[]

3.4.2 <Class / Object #2>

Class: Payment
3.4.2.1 Attributes
Payment ID (ie apple pay, Google Pay, PayPal, credit card)
Payment Date
Payment Amount

3.4.2.2 Functions
processPayment(movie details [])
refund Payment(Payment ID)

UML DIAGRAM 2:

PayMent
paymentID : int paymentDate : int paymentAmount : double paymentStatus : String
processPayment(paymentID : int, movieDetails : String[]) : boolean refundPayment(paymentID : int, movieDetails : String[]) : boolean

3.5 Non-Functional Requirements

3.5.1 Performance

95% of ticket booking transactions shall complete within 2 seconds. Payment processing shall respond within 3 seconds under normal load. The system shall support at least 500 concurrent users without degradation.

3.5.2 Reliability

The system shall have a mean time between failures (MTBF) of at least 30 days. All transactions shall be logged and recoverable in case of failure. Automatic retry mechanisms shall be in place for transient payment gateway failures. In case of loss of account, we will provide help in recovering accounts.

3.5.3 Availability

The system shall maintain an uptime of 99.9% measured monthly. Planned downtime for maintenance shall not exceed 30 minutes per month. The system shall provide informative messages during outages. The goal is to be always available.

3.5.4 Security

All payment data shall comply with PCI DSS standards. User authentication shall support multi-factor authentication (MFA). We will also implement two-factor authentication for users. Phone number for extra safety. Data transmission shall use end-to-end encryption (TLS 1.2). User passwords shall be stored using strong hashing algorithms.

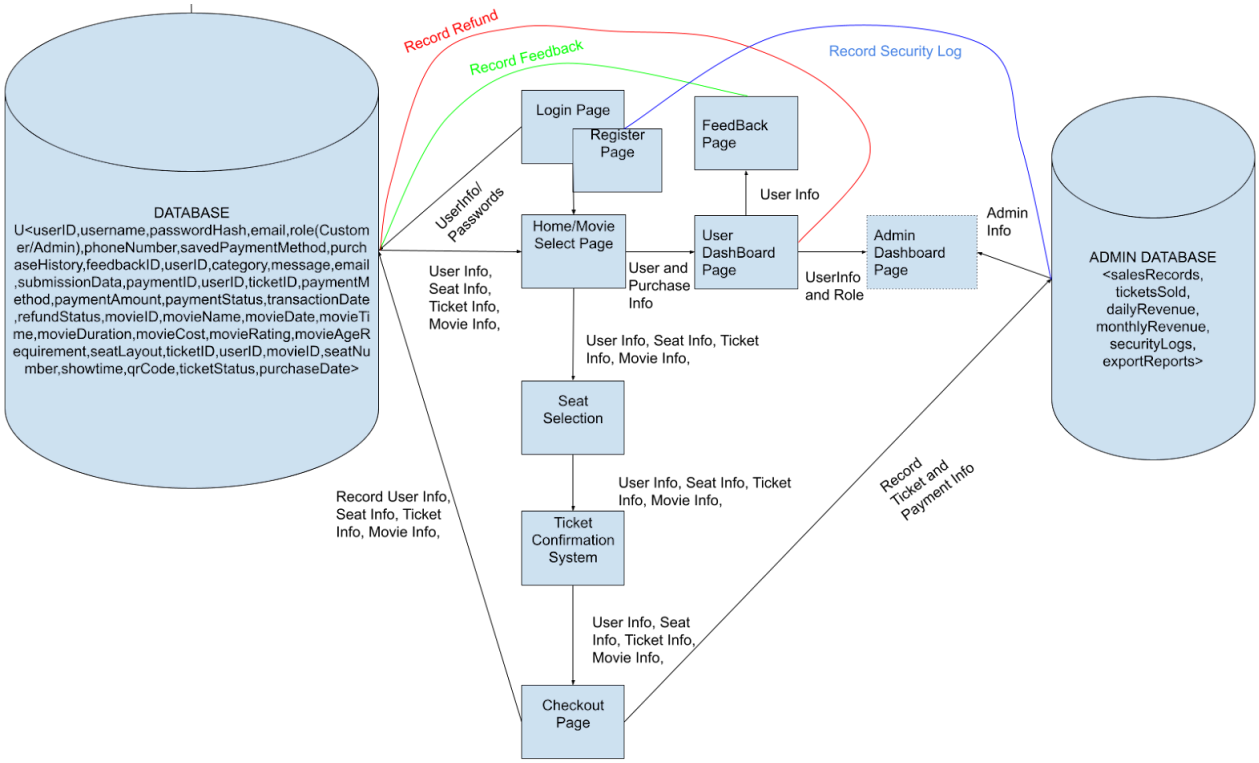
3.5.5 Maintainability

System design shall allow deployment of patches without downtime. Codebase shall be modular with a target of 80 % unit test coverage. Documentation shall be updated within 2 working days after any change.

3.5.6 Portability

Our web ticketing application will be available on web browsers such as Chrome, Firefox, Opera, etc. Additionally, we will also develop an app for customers.

3.6 ARCHITECTURAL DIAGRAM



PAGES/NODES

1. Home Page/ Movie selection Screen

Browse movies
View showtimes
Search/filter movies
List of movies
Select movie
View details (rating, age, time)

2. Login Page/Register

Username
Password
MFA (optional)
Name
Email
Password
Confirm password (record these in database and use the database to authenticate)

3. Seat Selection Page

Seat layout (available / taken)
Choose seat(s)

4. Ticket Confirmation Page

Ticket ID
QR code / barcode
Print ticket
Download ticket

5. Checkout / Payment Page

Display Price and all information record information in databases

6. User Dashboard (My Account Page)

View purchase history
View tickets
Manage payment methods

Request refund

7. Feedback / Bug Report Page

Submit feedback

8. Admin Dashboard Page

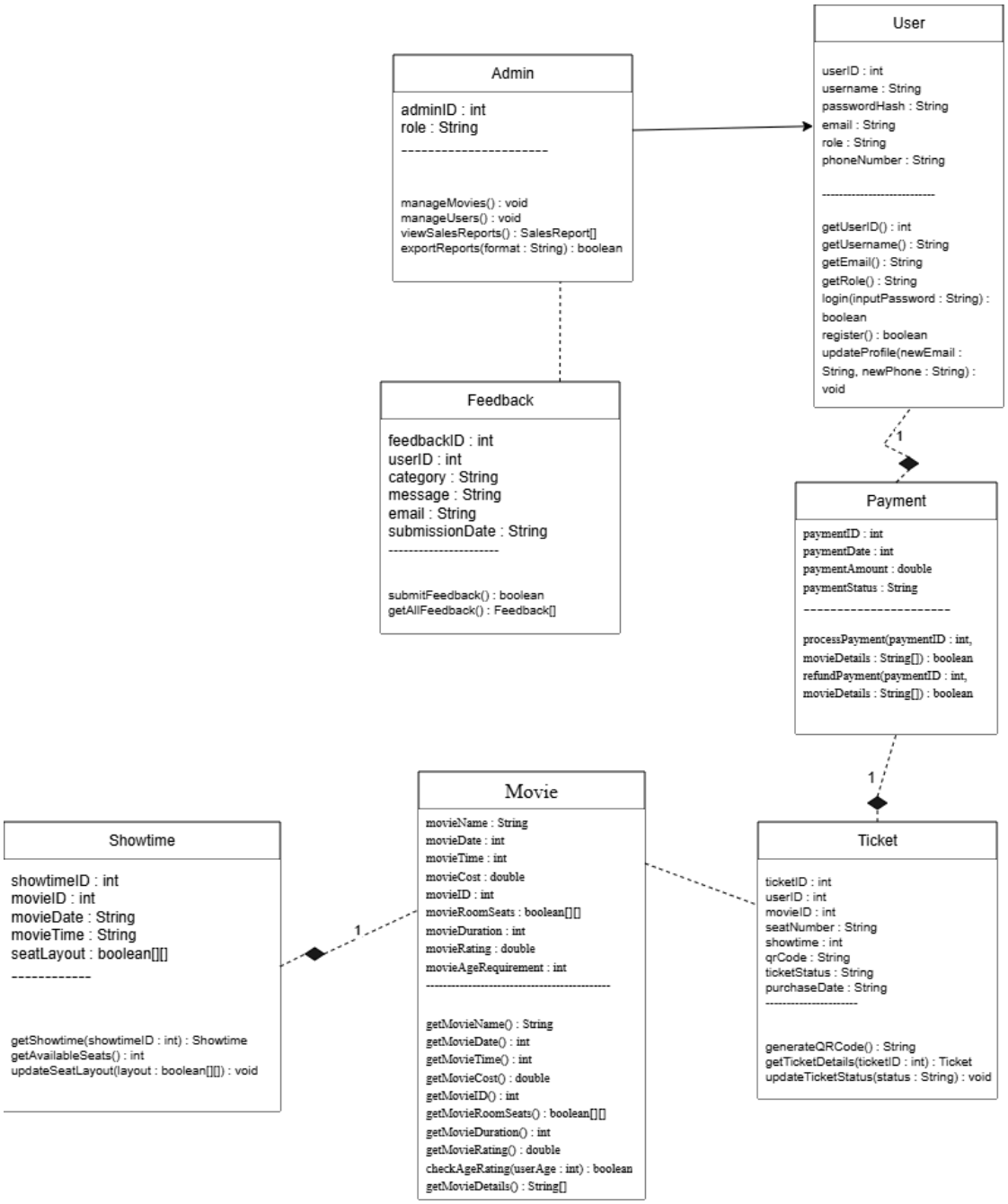
View ticket sales
Manage movies
Manage users
Export reports
View feedback

DATA BASE
userID
username
passwordHash
email
role (Customer/Admin)
phoneNumber
savedPaymentMethod
purchaseHistory
feedbackID
userID
category
message
email
submissionData
paymentID
userID
ticketID
paymentMethod
paymentAmount
paymentStatus
transactionDate
refundStatus
movieID
movieName
movieDate
movieTime
movieDuration
movieCost

movieRating
movieAgeRequirement
seatLayout
ticketID
userID
movieID
seatNumber
showtime
qrCode
ticketStatus
purchaseDate

ADMIN DATABASE
salesRecords
ticketsSold
dailyRevenue
monthlyRevenue
securityLogs
exportReports

3.7 UML CLASS DIAGRAM



KEY:
arrow: inheritance
Dotted line: association
Diamond: composition (child object cannot exist without parent object)

3.8 Class Descriptions

- User Class
  The User Class represents a customer who interacts with the ticketing system. Users can create accounts, browse movies, purchase tickets and manage their personal information.
  - Attributes
    - userID : int
    - username : String
    - passwordHash : String
    - email : String
    - role : String
    - phoneNumber : String
  - Operations
    - getUserID() : int
    - getUsername() : String
    - getEmail() : String
    - getRole() : String
    - login(inputPassword : String) : boolean
      Authenticates the user using the stored password hash
    - register() : boolean
      Creates a new user account in the database
    - updateProfile(newEmail : String, newPhone : String) : void
      Updates user contact information

- Admin Class
  The Admin class represents employees or managers who have administrative control over the system.
  - Attributes
    - adminID : int
    - role : String
  - Operations
    - manageMovies() : void
      Allows administrators to add, remove, or update movie listings
    - manageUsers() : void
      Allows administrators to view or manage user accounts
    - viewSalesReports() : SalesReport[]
      Allows administrators to review ticket sales data
    - exportReports(format : String) : boolean
      Exports reports in formats such as CSV or PDF.

- Movie Class
  The Movie class stores information about each movie being shown at the theater.
  - Attributes
    - movieName : String
    - movieDate : int

        - movieTime : int
        - movieCost : double
        - movieID : int
        - movieRoomSeats : boolean[][]
        - movieDuration : int
        - movieRating : double
        - movieAgeRequirement : int
    - Operations
        - getMovieName() : String
        - getMovieDate() : int
        - getMovieTime() : int
        - getMovieCost() : double
        - getMovieID() : int
        - getMovieRoomSeats() : boolean[][]
        - getMovieDuration() : int
        - getMovieRating() : double
        - checkAgeRating(userAge : int) : boolean
          Checks whether a user meets the age requirement for the movie.
        - getMovieDetails() : String[]
          Returns detailed movie information.

- Showtime Class
  The Showtime class represents a scheduled showing of a movie.
    - Attributes
        - showtimeID : int
        - movieID : int
        - movieDate : String
        - movieTime : String
        - seatLayout : boolean[][]
    - Operations
        - getShowtime(showtimeID : int) : Showtime
          Returns the showtime details
        - getAvailableSeats() : int
          Returns the number of available seats
        - updateSeatLayout(layout : boolean[][]) : void
          Updates the seating arrangement after ticket purchases

- Ticket Class
  The Ticket class represents a purchased movie ticket.
    - Attributes
        - ticketID : int
        - userID : int
        - movieID : int
        - seatNumber : String

        - showtime : int
        - qrCode : String
        - ticketStatus : String
        - purchaseDate : String
    - Operations
        - generateQRCode() : String
          Creates a unique QR code used for theater entry
        - getTicketDetails(ticketID : int) : Ticket
          Returns ticket information
        - updateTicketStatus(status : String) : void
          Updates ticket status such as used, refunded, or valid

- Payment Class
  The Payment class manages financial transactions for ticket purchases.
    - Attributes
        - paymentID : int
        - paymentDate : int
        - paymentAmount : double
        - paymentStatus : String
    - Operations
        - processPayment(paymentID : int, movieDetails : String[]) : boolean
          Processes the payment transaction through the payment provider
        - refundPayment(paymentID : int, movieDetails : String[]) : boolean
          Handles ticket refund processing

- Feedback Class
  The Feedback class stores customer feedback, bug reports, or feature requests.
    - Attributes
        - feedbackID : int
        - userID : int
        - category : String
        - message : String
        - email : String
        - submissionDate : String
    - Operations
        - submitFeedback() : boolean
          Stores feedback in the database
        - getAllFeedback() : Feedback[]
          Returns all feedback entries for administrators to review

3.9 Development Plan and Timeline

The project will be completed in several development phases.

- Phase 1 – System Design (1 Week)
  Develop system architecture and UML diagrams. Define database structure and system components.
- Phase 2 – Core System Development (2 Weeks)
  Implement user account management, movie browsing, and showtime selection features.
- Phase 3 – Ticket Purchasing System (1-2 Week(s))
  Develop seat selection, ticket generation, and payment processing functionality.
- Phase 4 – Administration Features (1 Week)
  Implement administrative tools for managing movies, users, and ticket sales reports.
- Phase 5 – Testing and Debugging (1 Week)
  Perform system testing to ensure that ticket purchases, payments, and data storage operate correctly.
- Phase 6 – Final Deployment (3-4 days)
  Finalize the system documentation and prepare the application for deployment.

3.10 Task Partitioning and Team Responsibilities

The project tasks are divided among the team members as follows:

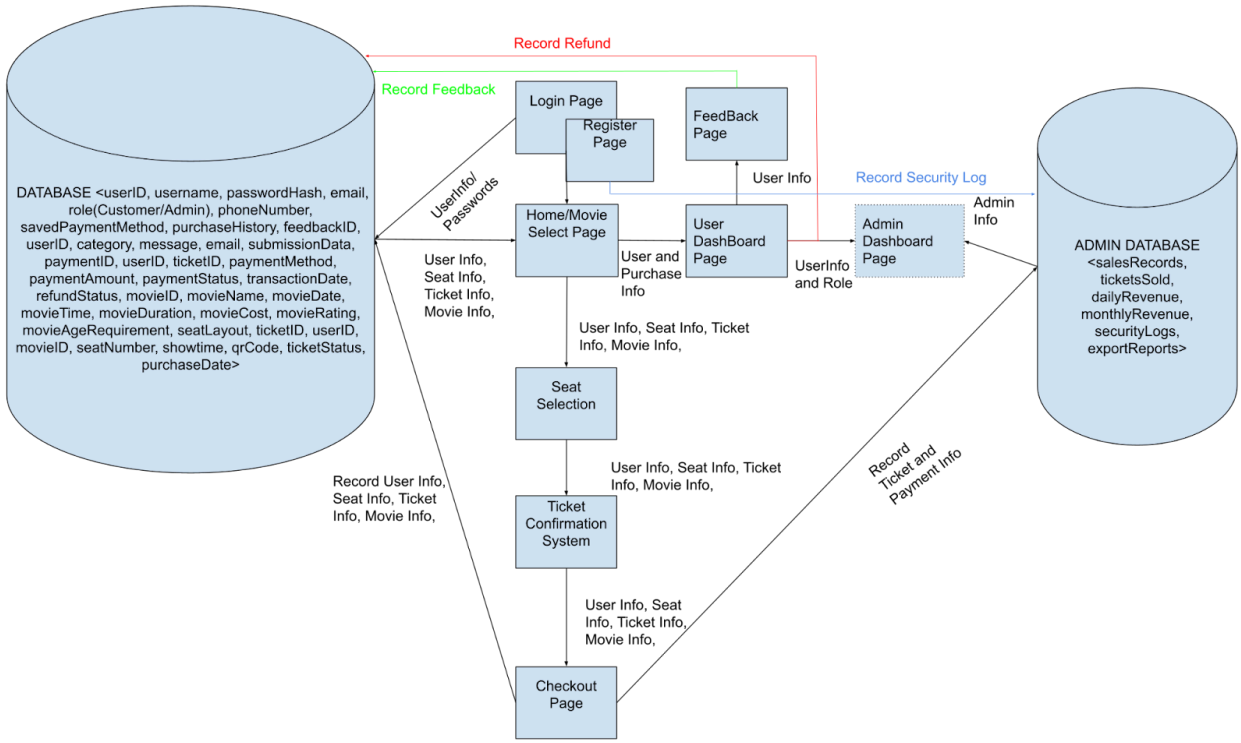
- Justin Nguyen -
  Responsible for system architecture design, database structure, and administrative system functionality.
- Christopher Lopez -
  Responsible for implementing the user interface, user account system, and ticket purchasing workflow.
- Yusuf Qargha -
  Responsible for payment processing integration, ticket generation, and system testing.

All team members will collaborate on documentation and final system integration

4. TEST PLAN

4.0 DESIGN SPECIFICATION SUMMARY:

Our UML diagram remains the same as it is well built and does not need any more improvements. As for our Software architectural design, we have decided to make some minor tweaks to make the document more readable and appear more professional.



Firstly, the free drawn lines are replaced with arrows that have straight lines and 90 degree angles. This makes the document easier to read on top of having a more professional feel. Additionally the database on the left (the main Database) has its variables spaced after each comma allowing for users to more easily distinguish variables from one another.

4.1 Overview

This section describes the overall verification and validation strategy for the Online Movie Theater Ticketing System (OMTTS). The goal of testing is to ensure that the system is both correctly implemented and meets user requirements.

- Verification: Ensures the system is built correctly according to the design (e.g., class behavior, logic correctness).
- Validation: Ensures the system meets user needs (e.g., correct ticket purchasing flow, usability).

Testing will be conducted throughout the development lifecycle and will include:

- Unit Testing
- Integration Testing
- System Testing

The system will be tested at multiple levels: unit, integration, and system testing, ensuring both individual components and the entire system function correctly together.

4.2 Testing Strategy

4.2.1 Black-Box Testing (Functional Testing)

Black-box testing focuses on validating system functionality based purely on inputs and outputs, without knowledge of internal implementation.

This approach will be used to verify:

- Correct behavior of user-facing features
- Proper handling of valid and invalid inputs
- Compliance with functional requirements and use cases

Used for:

- Ticket purchasing
- Refund system
- Account creation
- Feedback submission

This method is particularly useful for testing use case flows, such as ticket purchasing, refunds, and account creation.

4.2.2 White-Box Testing (Structural Testing)

White-box testing focuses on the internal logic and structure of the code. This ensures that all possible execution paths are tested and that no hidden bugs remain.

This approach will:

- Validate internal logic and decision-making processes
- Ensure all branches and conditions are executed at least once
- Detect hidden logical errors that may not appear in black-box testing

Coverage goals:

- Statement coverage: Every line of code is executed
- Branch coverage: Every decision path (if/else) is tested
- Condition coverage: All logical conditions are evaluated

Used for:

- Payment processing logic
- Seat selection updates
- Age verification checks

4.2.3 Test Data Selection

Effective test data selection ensures that test cases cover a wide range of possible inputs.

The following techniques will be used:

- Partition Testing: Inputs are divided into valid and invalid groups (e.g., valid vs invalid payment info)
- Boundary Value Testing: Focuses on edge cases such as:
    - Minimum age requirement
    - Last available seat
    - Maximum ticket purchase limits
- Random Testing: Random inputs are used to test general system robustness

This combination ensures both common and edge-case scenarios are properly tested.

4.3 Test Levels

4.3.1 Unit Testing

Unit testing focuses on testing individual components (classes and methods) in isolation to ensure they function correctly.

Each class in the system will be tested independently:

- User class: login validation, account creation, profile updates
- Movie class: retrieving movie details, age restriction checks
- Payment class: processing payments and refunds
- Ticket class: generating tickets and updating ticket status
- Feedback class: validating and storing user feedback

Mock data and controlled inputs will be used to simulate real scenarios. Unit testing helps identify bugs early in development before components are integrated.

4.3.2 Integration Testing

Integration testing verifies that multiple components work correctly when combined.

This level of testing focuses on:

- Data flow between modules
- Interface correctness
- Interaction between system components

Includes:

- Ensuring that successful payment triggers ticket generation
- Verifying seat availability updates after a purchase
- Confirming that feedback submissions are stored in the database

Integration testing ensures that dependencies between components do not introduce errors.

4.3.3 System Testing

System testing evaluates the complete, fully integrated system to ensure it meets all requirements.

This includes:

- End-to-end testing of user workflows
- Performance testing under expected load
- Security and reliability testing

Key scenarios tested:

- Full ticket purchase process from login to confirmation
- Refund processing workflow
- Admin dashboard operations
- System behavior under high user traffic

System testing ensures the application behaves correctly in real-world conditions.

4.4 Test Cases

TestCaseId	Component	Priority	Description/test summary	pre-requisites	test-steps	Expected result	Actual result	status	Test executed by:
UL:_1	User Class	P0	Verify user login succeeds with the input of the correct username/ password	User account is already created and exists in the database with a valid username and password	1: Enter the correct username 2: Enter the correct password 3: Click login	User successfully logs in and should be prompted with the appropriate user dashboard	User logs in. User is prompted with the user dashboard	N/A	TESTER1
UL_2	User Class	P0	Verify user accounts can be successfully created with	Username and email do not already exist in the system.	1: Enter a valid username, password,	A new user has been recorded and created in	New account created successfully	N/A	TESTER1

			valid inputs for required fields.		and email into the input fields 2: submit registration	the system			
UL_3	Movie Class	P0	Verify age restriction successfully blocks an underage user from purchasing a restricted movie ticket	A ticket has been selected, and the total price has been successfully displayed	1: select an R-rated movie 2: The age rating of the user is checked by the system 3: attempt to purchase a ticket	The checkAgeRating method should return false, and the purchase should be blocked	Purchase of the ticket is denied due to the age restriction	N/A	TESTER1
IL_1	Payment Class	P0	Verify that payment can be successfully processed and trigger a successful ticket generation	User is logged in, has a movie selected, and has an appropriate seat selected	1: submit a valid payment 2: processPayment is done by the system 3: ticket is generated	Payment is successful, and a ticket is also successfully generated	A ticket with a QR code or hash is generated	N/A	TESTER2
IL_2	Showtimes + ticket class	P0	Verify that seat availability is correctly updated after successful ticket purchases	Showtime exists and the seat is available	1: select a seat 2: purchase a ticket 3: The seat layout should be updated by system	The purchased seat becomes unavailable for the movie's seat layout	Seat is marked as unavailable	N/A	TESTER3
UL_4	User Class	P0	Verify login fails with incorrect password	User account exists with valid username and password	1: Enter correct username 2: Enter incorrect password 3: Click login	Login attempt is rejected and error message is displayed	Login denied with error message	N/A	TESTER1
UL_5	Movie Class	P0	Verify movie details are correctly retrieved	Movie exists in the system database	1: Call method to retrieve movie details using	Correct movie title, rating, duration, and description	Movie details displayed correctly	N/A	TESTER1

					movie ID	are returned			
UL_6	Payment Class	P0	Verify payment fails with invalid payment details	User is logged in and attempting payment	1: Enter invalid card/payment information 2: Submit payment	Payment is rejected and error message is shown	Payment failed and error displayed	N/A	TESTER2
UL_7	Ticket Class	P0	Verify ticket status updates correctly after cancellation	Ticket has been generated and exists in the system	1: Call method to cancel ticket	Ticket status is updated to “Cancelled”	Ticket status updated to cancelled	N/A	TESTER3
UL_8	Feedback Class	P0	Verify feedback submission rejects empty input	User is logged in	1: Leave feedback field empty 2: Submit feedback	System rejects submission and prompts user to enter valid feedback	Empty feedback not accepted	N/A	TESTER3

EXPLANATION:
These test cases are designed to cover the following levels and the following parts of our system:

Unit level:
Unit tests will focus on validating individual class methods independently to ensure that the core logic is implemented correctly.

These tests will cover:
User authentication logic
User account creation
Age restriction branch decisions

Integration level:
Integration tests verify that separate classes exchange data correctly and trigger dependent operations as expected.

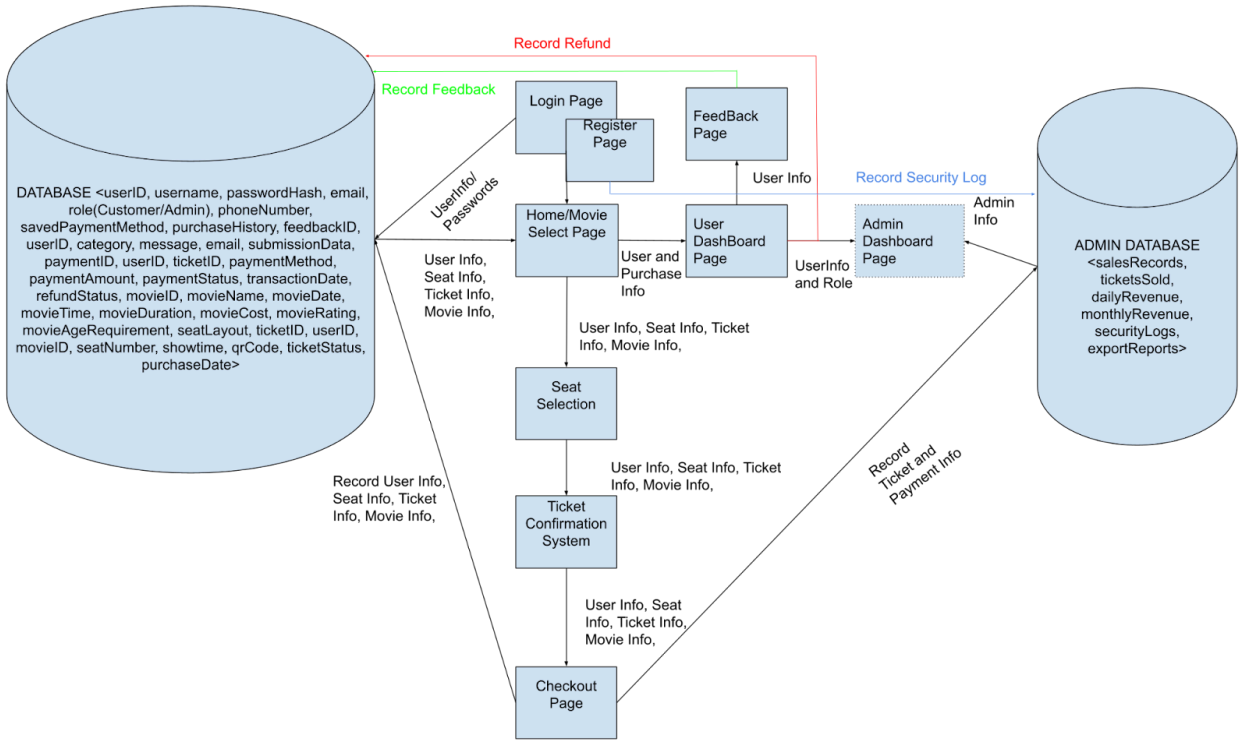
These tests will cover:
Payment interacting with ticket generation
Seat availability synchronization
Feedback persistence

System level:
System tests evaluate complete end-to-end workflows across multiple classes to ensure the full application behaves correctly under realistic usage scenarios.

These tests will cover:

Complete ticket purchase workflow
Refund workflow
Concurrent booking conflict
Invalid payment rejection

4.5 Software Architecture Diagram final design



We kept our software architectural diagram the same because we felt the design was strong, especially since we had previously revised it once to improve and fix earlier issues and fits our data management strategy. Our system uses two separate databases to organize data effectively and maintain security. The general user database stores information such as DATABASE <userID, username, passwordHash, email, role (Customer/Admin), phoneNumber, savedPaymentMethod, purchaseHistory, feedbackID, userID, category, message, email, submissionDate, paymentID, userID, ticketID, paymentMethod, paymentAmount, paymentStatus, transactionDate, refundStatus, movieID, movieName, movieDate, movieTime, movieDuration, movieCost, movieRating, movieAgeRequirement, seatLayout, ticketID, userID, movieID, seatNumber, showtime, qrCode, ticketStatus, purchaseDate>. In addition, we have a separate admin database that contains sensitive and restricted data only accessible to administrators, ADMIN DATABASE <salesRecords, ticketsSold, dailyRevenue, monthlyRevenue, securityLogs, exportReports>. This separation ensures better security in data management.

We use two databases to improve security, organization, and access control within our system. By separating the general user database from the admin database, we ensure that sensitive administrative data such as sales records, revenue, and security logs is only accessible to

authorized administrators and not regular users. This reduces the risk of unauthorized access or data breaches. Additionally, having two databases makes the system more organized and easier to manage, since user related data (like accounts, tickets, and payments) is kept separate from administrative data. It also allows us to implement role based access making sure each type of user only interacts with the data they are permitted to see and modify.

4.6 Data Management strategy:

Our strategy will notably be using two different databases. We chose this structure as it helps keep the system organized while also increasing security for information that needs to be more secure than others. We also use a database design as our application requires a reliable way of storing and managing structured information such as user accounts, tickets, payments, etc. A database-centered design allows our system to support everyday operations in the best way, as it provides persistent storage and is the best option for systems that use lots of structured information such as ours.

GENERAL USER DATABASE:
This is our first database, which stores operational data needed for normal system use. This notably includes USER INFORMATION and TRANSACTION INFORMATION. Examples of the stored data include userID, username, email, phonenumber, savedPaymentMethod, purchaseHistory, etc.
Essentially, this database will support the main user-facing parts of the system. For instance, the Home/Movie select page will need to read user info, seat info, ticket info, and movie info from our GENERAL USER DATABASE. The login page will also need to read userInfo/passwords.

ADMIN DATABASE:
This is our second database, which stores restricted and important information which should only be used by authorized administrators. This includes data such as salesRecords, ticketsSold, dailyRevenue, monthlyRevenue, securityLogs, and exportReports. The purpose of this database is to support the Admin Dashboard by allowing administrators to review business performance, monitor ticket sales, examine revenue trends, review system security activity, and generate reports. By keeping this info in a separate database, we strengthen access control as our more private data records for admins are not stored together with the general operational data used by customers.

SQL vs. NON-SQL AND OTHER TRADEOFFS :
For our movie ticketing system, we chose to use an SQL (relational database) design instead of a non-SQL design. The reason for this is that a relational database works best with a system that mainly works with structured data, like our system. A relational SQL approach fits this kind of system well as it’s designed to manage organized and structured data like ours, while also keeping this info accurate and organized.

We considered non-SQL as an alternative, as non-SQL could have provided more flexibility for storing some less structured data, such as feedback messages. However, our system does not primarily work with these types of data; thus, an SQL design is overall more beneficial for our system.

Lastly, we considered a hybrid design, where we can use a non-SQL database for items such as feedback or security logs. This could give flexibility for certain types of information, but would also make the system more complicated to design and maintain. We felt that the cost of increased effort required to maintain the system was not worth it.

In terms of tradeoffs for using a two-database system, the main tradeoff is that two SQL databases overall increase the complexity versus one database system. The complexity comes from now having to manage two separate systems for storage. However, the increase in complexity in this case is worth the increase in security for our system, as the risks we open ourselves to with a single database can be dangerous.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2